

## Requisitos funcionales:

1. **Administrar casos:** solicitar nombre, código y detective responsable. Al crear un nuevo caso, reemplazar el anterior y comenzar sin ubicaciones ni pistas.
2. **Registrar ubicaciones:** almacenar hasta cinco ubicaciones en un arreglo, incluyendo código, nombre, dirección, riesgo y estado. Validar que la posición exista y esté disponible.
3. **Consultar ubicaciones:** mostrar todas las registradas o una ubicación por su posición, comprobando si contiene null.
4. **Modificar y descartar ubicaciones:** actualizar su riesgo y estado, o eliminarlas dejando su posición en null.
5. **Registrar pistas:** almacenar en un ArrayList su código, descripción, tipo de evidencia, importancia y confiabilidad, sin permitir códigos repetidos.
6. **Consultar y buscar pistas:** mostrar todas las pistas o buscar una mediante su código, recorriendo el ArrayList.
7. **Modificar y eliminar pistas:** actualizar los datos de una pista existente o retirarla del ArrayList.
8. **Validar datos:** comprobar textos obligatorios, riesgo e importancia entre 1 y 10, y confiabilidad entre 0 y 100.
9. **Generar reportes:** mostrar cantidades de ubicaciones y pistas, espacios disponibles, ubicación de mayor riesgo, pistas de mayor importancia y confiabilidad, y promedio de importancia. Manejar las estructuras vacías.
10. **Manejar errores:** utilizar InputMismatchException, IllegalArgumentException, try-catch y finally, permitiendo continuar después de entradas incorrectas.
11. **Mostrar el menú:** incluir las trece opciones solicitadas y permitir salir del programa.

## Clases:

| Clase     | Propósito                                                                               |
|-----------|-----------------------------------------------------------------------------------------|
| Ubicación | Representar un lugar que se investigara y validar sus datos.                            |
| Pista     | Representar una evidencian encontrada y validar los datos.                              |
| Caso      | Guardar los datos generales de la investigación y administrar las ubicaciones y pistas. |
| Main      | Solicitar datos, mostrar el menú y comunicar los resultados al usuario.                 |

## Clase Ubicacion

### Atributos

| Atributo    | Tipo   | Propósito                                                   |
|-------------|--------|-------------------------------------------------------------|
| codigo      | String | Identificar la ubicación.                                   |
| nombre      | String | Guardar el nombre del lugar.                                |
| direccion   | String | Guardar la dirección o descripción del lugar.               |
| nivelRiesgo | int    | Representar el riesgo, entre 1 y 10.                        |
| estado      | String | Indicar su situación, por ejemplo, pendiente o investigada. |

### Métodos y constructor

| Método o constructor                                                                      | Retorno     | Propósito                                  |
|-------------------------------------------------------------------------------------------|-------------|--------------------------------------------|
| Ubicacion(String codigo, String nombre, String direccion, int nivelRiesgo, String estado) | Constructor | Crear una ubicación con datos válidos.     |
| getNivelRiesgo()                                                                          | int         | Consultar el riesgo para los reportes.     |
| actualizar(int nivelRiesgo, String estado)                                                | void        | Validar y modificar el riesgo y el estado. |
| getInformacion()                                                                          | String      | Devolver un texto con todos sus datos.     |

## Clase Pista

### Atributos

| Atributo           | Tipo   | Propósito                                          |
|--------------------|--------|----------------------------------------------------|
| codigo             | String | Identificar la pista y evitar registros repetidos. |
| descripcion        | String | Explicar qué se encontró.                          |
| tipoEvidencia      | String | Indicar el tipo de evidencia.                      |
| nivelImportancia   | int    | Representar la importancia, entre 1 y 10.          |
| nivelConfiabilidad | int    | Representar la confiabilidad, entre 0 y 100.       |

### Métodos y constructor

| Método o constructor                                                                                         | Retorno     | Propósito                                                 |
|--------------------------------------------------------------------------------------------------------------|-------------|-----------------------------------------------------------|
| Pista(String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad) | Constructor | Crear una pista con datos válidos.                        |
| getCodigo()                                                                                                  | String      | Consultar el código para buscarla y comprobar duplicados. |
| getNivelImportancia()                                                                                        | int         | Consultar la importancia para los cálculos.               |
| getNivelConfiabilidad()                                                                                      | int         | Consultar la confiabilidad para los cálculos.             |
| actualizar(String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad)           | void        | Validar y modificar los datos de la pista.                |
| getInformacion()                                                                                             | String      | Devolver un texto con todos sus datos.                    |

El código de la pista se conservará al modificarla, porque se utilizará para identificarla.

## Clase Caso

### Atributos

| Atributo | Tipo   | Propósito                              |
|----------|--------|----------------------------------------|
| nombre   | String | Guardar el nombre de la investigación. |

| Atributo             | Tipo             | Propósito                                  |
|----------------------|------------------|--------------------------------------------|
| codigo               | String           | Identificar el caso.                       |
| detectiveResponsable | String           | Guardar el nombre del detective.           |
| ubicaciones          | Ubicacion[]      | Almacenar hasta cinco ubicaciones.         |
| pistas               | ArrayList<Pista> | Almacenar una cantidad variable de pistas. |

## Métodos y constructor

| Método o constructor                                                                                                  | Retorno     | Propósito                                                       |
|-----------------------------------------------------------------------------------------------------------------------|-------------|-----------------------------------------------------------------|
| Caso(String nombre, String codigo, String detectiveResponsable)                                                       | Constructor | Crear el caso con sus estructuras vacías.                       |
| validarPosicion(int posicion)                                                                                         | void        | Comprobar que el índice esté entre 0 y 4. Será privado.         |
| registrarUbicacion(int posicion, Ubicacion ubicacion)                                                                 | void        | Guardar una ubicación en una posición disponible.               |
| consultarUbicaciones()                                                                                                | String      | Devolver las ubicaciones registradas y sus posiciones.          |
| obtenerUbicacion(int posicion)                                                                                        | Ubicacion   | Obtener una ubicación; devolver null si la posición está vacía. |
| modificarUbicacion(int posicion, int nivelRiesgo, String estado)                                                      | void        | Actualizar una ubicación existente.                             |
| descartarUbicacion(int posicion)                                                                                      | void        | Eliminar una ubicación asignando null.                          |
| registrarPista(Pista pista)                                                                                           | void        | Agregar una pista si su código no está registrado.              |
| consultarPistas()                                                                                                     | String      | Devolver la información de todas las pistas.                    |
| buscarPista(String codigo)                                                                                            | Pista       | Buscar una pista; devolver null si no existe.                   |
| modificarPista(String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad) | void        | Actualizar una pista existente.                                 |
| eliminarPista(String codigo)                                                                                          | void        | Localizar y eliminar una pista.                                 |
| generarReporte()                                                                                                      | String      | Calcular y devolver el reporte completo de la investigación.    |

## Clase Main

### Atributos

| Atributo   | Tipo    | Propósito                                                           |
|------------|---------|---------------------------------------------------------------------|
| scanner    | Scanner | Leer los datos ingresados por el usuario. Será privado y estático.  |
| casoActual | Caso    | Guardar el caso que se está administrando. Será privado y estático. |

### Métodos

| Método o constructor       | Retorno | Propósito                                             |
|----------------------------|---------|-------------------------------------------------------|
| main(String[] args)        | void    | Iniciar el programa y ejecutar las opciones del menú. |
| mostrarMenu()              | void    | Mostrar las trece opciones disponibles.               |
| crearCaso()                | Caso    | Solicitar los datos y devolver un caso nuevo.         |
| leerTexto(String mensaje)  | String  | Mostrar una solicitud y leer una línea de texto.      |
| leerEntero(String mensaje) | int     | Solicitar un entero y manejar entradas incorrectas.   |

## 3. Propiedad que utilizara un arreglo básico

El atributo ubicaciones de Caso utilizara un arreglo de objetos Ubicación con tamaño cinco:

```
Ubicaciones = new Ubicaciones[5];
```

Sus índices serán 0, 1, 2, 3 y 4. Al crearlo, las cinco posiciones contendrán null.

## 4. Propiedad que utilizara un ArrayList

El atributo pistas de Caso utilizara:

```
Pistas = new ArrayList <Pista>();
```

Almacenara objetos Pista y permitirá aumentar o reducir la cantidad de registros. Se utilizara ArrayList directamente.

## 5. Modificadores de visibilidad

Los atributos serán private. Los constructores y métodos que deben utilizar otras clases serán public.

Los métodos auxiliares, como validar Posición, serán private. Así las modificaciones se realizarán mediante métodos que revisen los datos antes guardados,

## 6 Parámetros requeridos por los métodos

Los parámetros aparecen en las tablas anteriores. Se utilizaran:

- Datos de texto y valores numéricos para crear los objetos.
- Una posición int para operar sobre el arreglo.
- Un código String para localizar pistas.
- Los nuevos datos para las modificaciones.
- Un objeto Ubicación o Pista para registrarlo.

## 7 Inicialización y validación de objetos

Los objetos recibirán sus valores mediante constructores. El constructor de Caso también creará el arreglo de cinco ubicaciones y el ArrayList vacío.

Antes de crear o modificar objetos se comprobará que:

- Los campos de texto no sean nulos ni estén vacíos.
- El riesgo este entre 1 y 10.
- La importancia este entre 1 y 10.
- La confiabilidad este entre 0 y 100.
- La posición del arreglo sea válida.
- La posición este disponible al registrar una ubicación.
- El código de una nueva pista no este repetido.

## 8 Comprobación de posiciones vacías

Primero se validará el índice. Después se comprobara el contenido:

```
if (ubicaciones[posicion] != null) {  
    // La posición contiene una ubicación.  
} else {  
    // La posición está vacía.  
}
```

Al recorrer el arreglo también se hará esta comprobación antes de llamar a los métodos de cada ubicación.

## 9 Búsqueda modificación y eliminación de pistas

Para buscar una pista se recorrerá el ArrayList con un for, comparando el código solicitado con el de cada objeto mediante equalsIgnoreCase().

Si se encuentra, se devolverá el objeto. Si no se encuentra se devolverá null.

Para modificarla se utilizará la búsqueda y después se llamará a actualizar(). Para eliminarla se buscará el objeto y una vez terminando el recorrido de búsqueda se utilizará:

```
Pistas.remove(pistaEncontrada);
```

## 10 Excepciones y manejo de errores

| Situación                                                                    | Manejo                                                                                                                  |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| El usuario escribe letras cuando se solicita un entero.                      | Capturar InputMismatchException dentro de leerEntero(), limpiar la entrada con nextLine() y volver a solicitar el dato. |
| Se ingresa un riesgo, importancia o confiabilidad fuera del rango permitido. | Lanzar IllegalArgumentException desde Ubicacion o Pista.                                                                |
| Se ingresan textos vacíos.                                                   | Lanzar IllegalArgumentException al validar los datos.                                                                   |
| Se intenta usar una posición fuera del arreglo.                              | Detectarlo en validarPosicion() y lanzar IllegalArgumentException antes de acceder al arreglo.                          |
| Se registra una ubicación en una posición ocupada.                           | Lanzar IllegalArgumentException desde Caso.                                                                             |
| Se modifica o elimina una ubicación vacía.                                   | Comprobar null e informar mediante IllegalArgumentException.                                                            |
| Se registra una pista con código repetido.                                   | Lanzar IllegalArgumentException desde Caso.                                                                             |
| Se modifica o elimina una pista inexistente.                                 | Comprobar el resultado de la búsqueda y lanzar IllegalArgumentException.                                                |